1. Vom face reducerea $\overline{\texttt{HALT}} \leq f$; știm de la curs că $\overline{\texttt{HALT}} \notin R$.

   $$\overline{\texttt{HALT}} \leq_m f \stackrel{\text{def}}{=} \exists t : \Sigma^* \to \Sigma^*, \text{ a.î. } \begin{cases} t \text{ e computabilă} \\ \forall (M, w), \quad \overline{\texttt{HALT}}(enc((M, w))) = TRUE \Leftrightarrow f(t(enc((M, w)))) = TRUE \end{cases}$$

   Transformarea t produce următoarele două mașini M_1 și M_2:

$M_1[v]$:
   1. rulează $M[w]$
   2. tranziționează în H

$M_2[v]$:
   1. ciclează la infinit

   t trebuie doar să producă definiția acestor mașini, deci e computabilă.

   Demonstrăm echivalența, cu două implicații:

   "$\Rightarrow$": $\overline{\texttt{HALT}}(enc((M, w))) = TRUE \Rightarrow M[w]$ doesn't halt $\Rightarrow \forall v \in \Sigma^* M_1[v]$ doesn't halt , $M_2[v]$ doesn't halt $\Rightarrow$ $M_1[enc(M_2)]$ doesn't halt , $M_2[enc(M_1)]$ doesn't halt $\Rightarrow f(enc(M_1, M_2)) = TRUE$.

   "$\Leftarrow$": $f(enc(M_1, M_2)) = TRUE \Rightarrow M_1[enc(M_2)]$ doesn't halt , $M_2[enc(M_1)]$ doesn't halt $\Rightarrow$ $M_1[enc(M_2)]$ nu ajunge la linia 2 $\Rightarrow M[w]$ doesn't halt $\Rightarrow \overline{\texttt{HALT}}(enc((M, w))) = TRUE$.

2. Alegem două funcții reprezentative:

   $$f(n) \in \Omega(n^3) \tag{1}$$
   $$g(n) \in O(n^2) \tag{2}$$

   Expandăm definițiile pentru a obține:

   $$\exists c_1 \in \mathbb{R}_+, \exists n_1 \in \mathbb{N} \text{ a.î. } \forall n \geq n_1 : \quad 0 \leq c_1 n^3 \leq f(n) \tag{3}$$
   $$\exists c_2 \in \mathbb{R}_+, \exists n_2 \in \mathbb{N} \text{ a.î. } \forall n \geq n_2 : \quad 0 \leq g(n) \leq c_2 n^2 \tag{4}$$

   Împărțim $f(n)$ la $g(n)$ și $c_1 n^3$ la $c_2 n^2$:

   $$\frac{f(n)}{g(n)} \geq \frac{c_1 n^3}{c_2 n^2} = \frac{c_1}{c_2} n$$

   Notăm o nouă constantă $c = \frac{c_1}{c_2}$. Putem rescrie inegalitatea:

   $$\frac{f(n)}{g(n)} \geq cn$$

   Deci $\frac{f(n)}{g(n)} \in \Omega(n)$.

3. $T(n) = T(\frac{n}{2}) + 2T(\frac{n}{4}) + n$.
   Expandăm definiția a ce înseamnă $T(n) \in O(n \log n)$:

   $$\exists c \in \mathbb{R}_+, \exists n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0 : \quad 0 \leq T(n) \leq cn \log n$$

   Cazuri de bază:

   - $n = 1$: $0 \leq 1 \leq 0$ – fals! $n = 1$ nu poate fi caz de bază pentru demonstrația prin inducție! Trebuie să începem de la $n = 2$; pentru că definiția recursivă a $T(4)$ implică valoarea lui $T(1)$, e nevoie să luăm $T(4)$ caz de bază în inducție.
   - $n = 2$: $0 \leq 1 \leq c2 \Rightarrow c \geq \frac{1}{2}$

- $n = 3$: $0 \leq 1 \leq c3\log 3 \Rightarrow c \geq \frac{1}{3\log 3}$
- $n = 4$: $0 \leq 7 \leq c8 \Rightarrow c \geq \frac{7}{8}$

Pas inductiv Presupunem:

- $0 \leq T\left(\frac{n}{2}\right) \leq c\left(\frac{n}{2}\log\frac{n}{2}\right)$ (1).
- $0 \leq T\left(\frac{n}{4}\right) \leq c\left(\frac{n}{4}\log\frac{n}{4}\right)$ (2).

Trebuie să arătăm că: $0 \leq T(n) \leq cn\log n$.
$(1) + 2 \times (2) + n$:

$$
\begin{aligned}
n \leq T(n) &\leq c\left(\frac{n}{2}\log\frac{n}{2}\right) + 2c\left(\frac{n}{4}\log\frac{n}{4}\right) + n && (5)\\
&= c\frac{n}{2}(\log n - 1) + 2c\frac{n}{4}(\log n - 2) + n && (6)\\
&= c\frac{n}{2}\log n - c\frac{n}{2} + c\frac{n}{2}\log n - cn + n && (7)\\
&= cn\log n + n(1 - c - \frac{c}{2}) && (8)\\
&= cn\log n + n(1 - \frac{3c}{2}) && (9)\\
&\leq cn\log n && (10)
\end{aligned}
$$

$\forall n \geq 5, \quad n \geq 0$, deci inegalitatea $0 \leq T(n)$ e adevărată.
Ultima inegalitate ține doar dacă

$$1 - \frac{3c}{2} \leq 0 \Rightarrow c \geq \frac{2}{3}$$

Orice valoare supraunitară pentru c (e.g. $c = 1$) satisface cu siguranță toate constrângerile, deci $T(n) \in O(n\log n)$.

4. Implementarea lui `pushMin` în pseudocod:

```
pushMin(x):
    while stack.top() < x:
        stack.pop()

    stack.push(x)
```

Astfel, mereu elementul din vârf e cel mai mic, deci `popMin` poate consta într-un singur `pop`.

Fiecare operație de `pushMin` trebuie să acopere operația de `push`, un viitor `top` și un `pop` (pentru când elementul va fi scos din stivă de un alt `pushMin`, sau de `popMin`). Deasemenea, orice `pushMin` (în afară de primul) inspectează cel puțin un element din stivă, deci trebuie să acopere costul unui `top`. Deci trebuie să contribuie cu 4 credite.

Secvența "worst-case" constă în inserarea a $n-1$ numere descrescătoare, urmată de inserarea unui număr mai mic ca toate celelalte (de fapt, orice secvență care se termină în inserarea unui număr mai mic ca toate elementele deja prezente în stivă, are același cost).

$$\sum_{i=1}^{n} c_i = 2(n-1) + (2(n-1)+1) = 4n - 3 \leq 4n = \sum_{i=1}^{n} \hat{c}_i$$

5. Caz de bază ($t = Nil$): $size(mirror(Nil)) \overset{M1}{=} size(Nil)$.
   Presupunem $size(l) = size(mirror(l))$ și $size(r) = size(mirror(r))$.
   Demonstrăm că $size(Node(k,l,r)) = size(mirror(Node(k,l,r)))$:

$$
\begin{aligned}
size(Node(k,l,r)) &\overset{S2}{=} 1 + size(l) + size(r) && (11)\\
&\overset{I.I.}{=} 1 + size(mirror(l)) + size(mirror(r)) && (12)\\
&\overset{=}{1} + size(mirror(r)) + size(mirror(l)) && (13)\\
&\overset{S2}{=} size(Node(k, mirror(r), mirror(l))) && (14)\\
&\overset{M2}{=} size(mirror(Node(k,l,r))) && (15)
\end{aligned}
$$